
UNIT 11 ADVANCE OBJECT DESIGN

Structure

- 11.0 Introduction
 - 11.1 Objectives
 - 11.2 Control and its Implementation
 - 11.2.1 Control as a State within Program
 - 11.2.2 Control as a State Machine Engine
 - 11.2.3 Control as Concurrent Task
 - 11.3 Inheritance Adjustment
 - 11.4 Association: Design
 - 11.5 Object Representation
 - 11.6 Design Optimization
 - 11.7 Design Documentation
 - 11.8 Summary
 - 11.9 Solutions/Answers to Check Your Progress
 - 11.10 References/Further Reading
-

11.0 INTRODUCTION

Object Orient arises to address various problems that can be grouped and are called “Software Crisis”. Object-oriented approach provides a solution using real-life principles. History of OOAD includes, until 1997 the following nomenclature has been used:

Object Oriented Software Engineering –Ivar Jacobson

Object Modeling Technique- James Rumbaugh

Object Oriented Analysis and Design-Grady Booch

Given the problem domain, the foremost task of OOAD is to break it into various processes and modularise the behaviour into function. Object-oriented approach is used for the identification of objects from the problem domain and use them in problem solving.

An object is an important element of Object Oriented Technique and can be represented as:

- a physical entity such as car, truck etc.,
- a conceptual entity like any chemical reaction or process or the software entity, for example, linked list, stacks etc.

Object can also be defined as an instance of a class which is identifiable thing and linked with classes that are part of any application. Every object contains associated Data (attribute) and has some features implemented in the form of functions. By using the concept of object-orientation an implementation/program can be decomposed into a number of objects. Figure 11.1 shows the structure of objects.

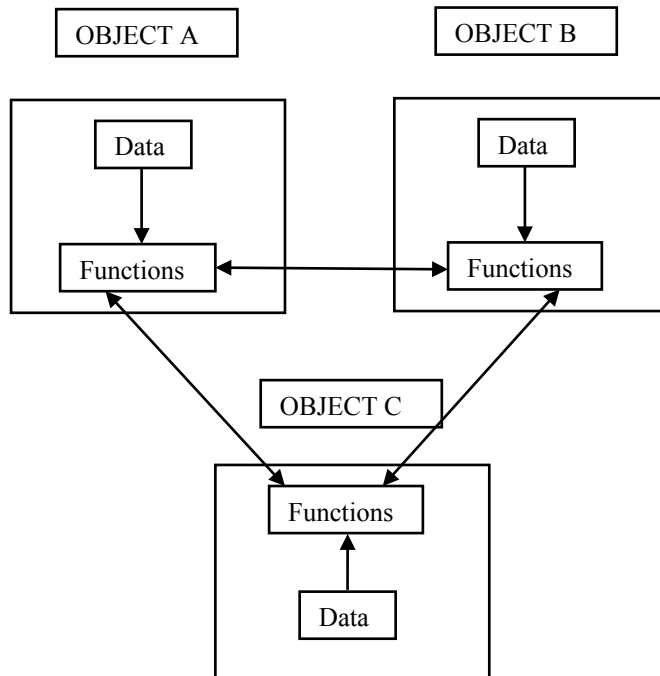


Figure 11.1: Objects and their Interaction

In this unit, we will learn concepts and diagrams used in object design.

11.1 OBJECTIVES

After going through this Unit, you should be able to :

- Explain OOD characteristics and advantages,
- Explain the three models of OMT,
- Use Implementation of control,
- Do Inheritance Adjustment,
- Describe Design Association,
- Determine Object Representation,
- Explain objects representation, and
- Apply different ways to optimize OOD process.

11.2 CONTROL AND ITS IMPLEMENTATION

To manage the activities of any system, many controls are used. Control means how the program change state from one to another after giving the input. In this section, we will discuss object's states and representation of their behaviour.

State Machine- Object behaviour can be modelled by using static machine modeling. Any event that affects the object state, causing some occurrence of actions. State Transition Diagram(STD) is used to represent it graphically.

Though we have already learned about terms and concepts like state, activity, transition etc., let us revise some of these important terms used in STD:

- **State** - A curved edge rectangle is used to represent states in STD. During its lifecycle object goes through different states based on its attributes and function. Each state has a name and action which may happen while entering or exiting the particular state, i.e., changes that occur in a particular state but are internal to that state and do not result in a change of state. A state can be part of a big state and is known as sub-state of the big state, also, there is starting state and last state. An object's initial state is its start state, also known as the beginning state, and is shown with a black circle in a STD. The final state represents the last step in execution and is diagrammatically given as a black circle enclosed in another circle.
- **Transition**- The transition from one state to another is shown by an arrow from an old state to a new state. When an object satisfies certain applied conditions and after an event which performs various activities results in a new state.

Let us take an example of an Order Management System given in Figure 11.2. It starts from Idle state and reaches to Dispatch order state (taken its initial and final state). The different intermediate states that can be possible are sending order, selecting order, confirming order etc.

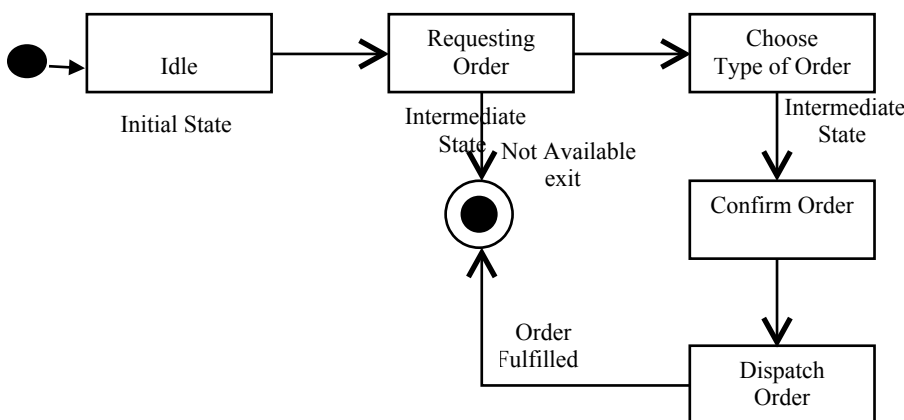


Figure 11.2: Order Management Process represented as State Transition Diagram

The various parts of transition are:

- Source State:** It is the initial state of the object before any transition.
- Events:** It changes the state of an object from the source to the intermediate state after it satisfies certain conditions. Examples include - mouse click, entering any key, a software or hardware interrupt etc.
- Condition:** If the required condition is satisfied, then it returns true and otherwise false; and no state change in state.
- Action:** It is an outcome because of transition. A series of actions represent a certain scenario. For example, in an ATM machine, an error message(action) may come because of the wrong entry, or cash withdrawal (action) occur if the condition is satisfied. It may execute while entering a state or exiting from a state.
- Final State:** It occurs as a result of the last transition and action.

11.2.1 Control as a State within Program

Control means how the program changes state from one to another after giving the input. As we know, there are many ways to show control in a program. Control can be represented as an iterative approach using a loop or as a conditional branch statement in the program, depending on the input read. If there are many nested loop in a code, then input is transferred from low to high level until some procedure can handle it. Various steps involved in changing the state transition diagram to code are mentioned below:

- i. First, detect the main and essential path- Generally, the flow of a program is sequential so identify the states which are intermediate from the initial state, and this provides the path for the sequential events in a program which are then converted into a programming statement of a program.
- ii. Check alternative routes from the main path that originates after satisfying some condition; these are written as a conditional statement later in the code.
- iii. To identify the loop or iteration code in a program, read the state transition diagram for all the circle coming back to the main branch; if these are several in number and not crossing each other nested loops can be generated with these backward jump path. Some backward paths will be implemented by means of *goto statement* in a program.
- iv. Unchecked can be a case of exception handling, and depending on the language used, it will get implemented by the usual status flag or error routines or exception handling statements or functions.

11.2.2 Control as a State Machine Engine

A state machine is a behaviour model. It consists of a finite number of states. It needs an external event that will cause branching to the next activity. In other words, a **state machine** is event-driven. One way of implementing control is to represent and execute the State machine engine. With this technique of defining classes and routines or stub it's easy to do advancement from Analysis model to prototype building.

For example, if a transition and action table is provided, then it's easy to form a class named "State Machine". Each object instance contains state variables and can call a state machine engine to determine the state and the required actions. Each object instance would contain its independent state variables but call on the state engine to determine the next state and action.

11.2.3 Control as a Concurrent Task

Concurrency is identified and represented in the dynamic model. Concurrency allows more than one object to receive events simultaneously and more than one activity to be executed simultaneously. Generally, conventionally, concurrency needs to be avoided in programming, and the sequential program approach is to be followed; however, parallelism has several advantages. The foremost reason to implement concurrency is, that

some tasks are inherently concurrent, and another reason might be the economical use of resources among concurrent tasks.

Construct used to express concurrency are specified below:

- For implementing interleaved concurrency, coroutines strategies may be used. Coroutines are organized at the same level.
- Another construct is the fork and join instruction. With fork, simultaneous execution of two concurrent programs of the same code is invoked.
- The sequential program may execute parallelly and interact just by passing messages or through variable sharing.
- Activities can be synchronized with the use of semaphores but they are very unstructured and may be replaced by monitor construct.
- Executing entity, "thread" with its associated data and other resources may be used to handle concurrency. Thread shares the same namespace, and each thread has its own resources and data, which allow concurrency within the process. Process is known as active object and has a thread of control.

Concurrency in objects are considered in various ways:

- Intra object concurrency occurs when several threads of the same process execute concurrently.
- Inter Object concurrency allows independent object to execute simultaneously.

If many objects received events concurrently, causing simultaneous execution of activities, but these objects are not interacting, then allow only one object per thread in a State Transition Diagram and a separate thread for other objects. If an individual object operation is concurrent, then split object into various threads as concurrency causes problems such as Deadlocks, disintegration of data etc., so it should be identified and resolved at the design level rather than at the implementation level. Concurrency in object should be fixed and implemented as a task in different programming languages or operating systems depending upon the developer's need.

Concurrency focuses on abstraction and synchronization. For example, in an automated system, concurrency is required for many problems, as it requires various events to handle or for problems that require a lot of computation that a single processor may not handle. The use of multitasking and multiprocessing can solve these problems. By using a single processor(Root) many independent actions may be handled. Each program has at least one thread and will require a system of multithreading. Multiple CPU systems allow concurrent Threads to run. Concurrency can also be classified as lightweight and heavyweight. Lightweight processes share a common address space, whereas heavyweight processes exist independently.

 Check Your Progress-1

1. Briefly explain the state diagram with the help of an example.

.....

.....

.....

2. Explain the concept of concurrency.

.....

.....

.....

.....

3. Draw State Transition Diagram of an ATM machine.

.....

.....

.....

.....

11.3 INHERITANCE ADJUSTMENT

Inheritance is the way to create new classes from the existing ones. The new class formed is called child/derived/subclass, and the already existing class is known as base/parent/superclass. Inheritance type includes: Single, multiple, hybrid, multilevel and hierarchical inheritance

With the progress of object design, one of the issues that arose was the adjustment of inheritance. The definition of various classes and operations can be adjusted for the adjustment of inheritance.

To increase inheritance, the following steps should be performed:

- a) Rearrangement of classes and its operations.
- b) Abstract common behaviour out of groups of clusters. If more than one class shares a common operation and attribute, then make use of abstraction and form a superclass which contains shared properties or features and leaves only specialized features in subclasses.
- c) Apply delegation to share implementation following readjustment causes an increase in inheritance.

a) Reorganize classes and their associated behaviour to increase the inheritance

Inheritance can be increased by making the various adjustments as mentioned below:

- Some operations may have optional arguments.
- Operations contain fewer arguments and are special cases of general arguments.
- Attributes with likenesses and different names in various operations can be transferred to a parent class so that all operations can access them.
- Inconsistent naming strategy for operations can be avoided.
- Some common operations can be included in the parent class and may be relevant for some base classes and irrelevant operations for others; hence, it is declared as no operations in them.

b) Abstract the common behaviour in classes

Abstraction results in modularity. If two or more classes have common behaviour, a new superclass will be fabricated, containing the common features of all sub-classes and the base classes only containing the specialised features. Such abstracted super classes can be identified, and a library of these reusable classes will be created and will be used in future projects. Abstract classes also improve extensibility. Figure 11.3 shows the abstract class shape.

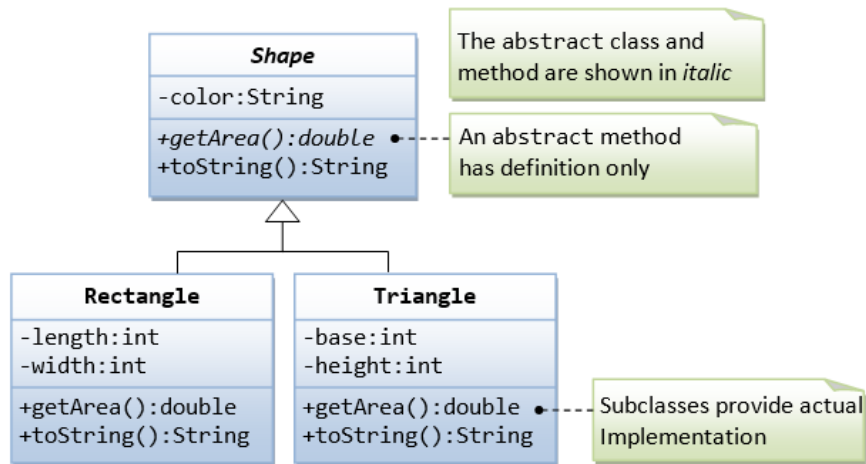


Figure 11.3: Abstract class and subclasses

c) Use delegation to share behaviour when inheritance is semantically invalid

Inheritance uses a transitive property where each subclass inherits attributes, associations and operations from their super class. Inheritance follows a relationship. Subclass inherits a superclass completely if operations of derived class override the similar operation of base class in that case it is said to be correct that the subclasses are specialization of the super classes. For example, consider the implementation of stack class (derived class), and a Linked list class is already available. The Linked list class is defined with five operations viz. Insert, Delete, Beg, End and Display and stack class with two operations push and pop as shown in Figure 11.4.

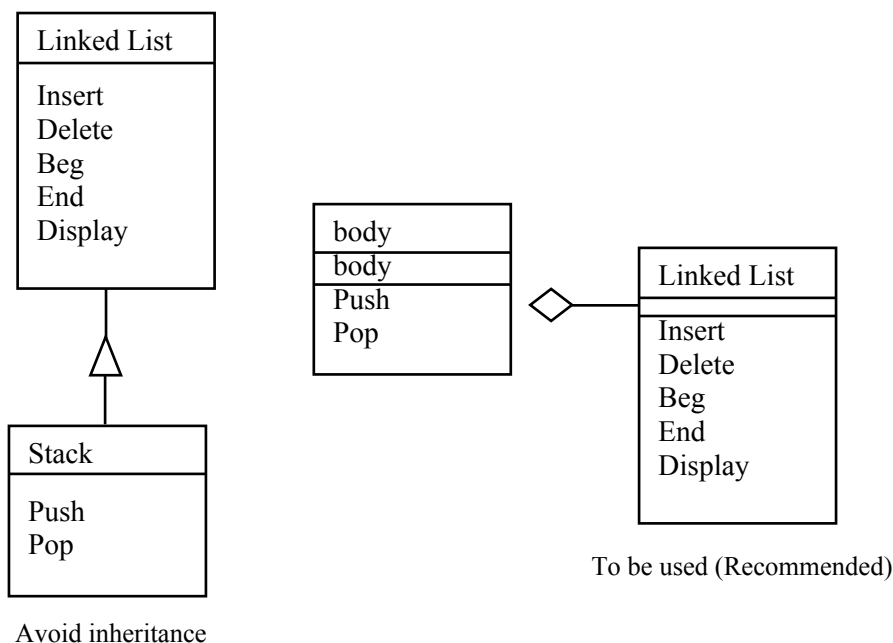


Figure 11.4: Implementation of Stack class by Inheritance(Left) and delegation(Right)

Now, if a Stack class inherit Linked List then it will also inherit, not needed operation of Linked List class. Another way to implement Stack is to delegate some features of Linked List class, for example, push can be implemented by delegation of Insert and end operation and pop by the delegation of Delete and end operation from Linked List class. Other operations are hidden and not to be used, as only required operations should be delegated, so in some cases like this, discourage inheritance and use delegation . Delegation is used where has a relationship is required.

11.4 ASSOCIATION DESIGN

Association is defined as the relationship that exists among objects of different classes. A link is used to represent this relationship among objects. The plan for executing of the association has been formulated in object modelling. The strategy can be a Global for all associations to be implemented or different techniques for individual associations.

Two object can be linked if there exists an association among their related classes. The link represent the path or the link between the instances of the classes. Association describes as a structural relationship among objects. It is represented as a solid line mainly directed, usually includes labels, and often specifies multiplicity and role names. Example relationship between a Person and Organisation is ahown in figure 11.5.

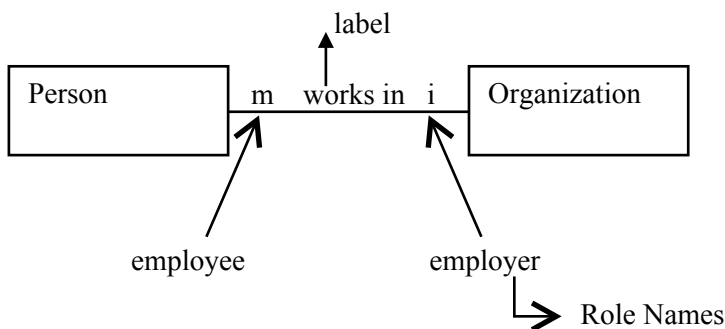


Figure 11.5 : Association of Person and Organization

Association modelling steps to be followed are:

- Data driven View: It is used to navigate between the objects of the classes, it is needed to show the association between two classes.
- Behaviour driven View: This allows for interaction between objects of different classes.
- Define multiplicity and roll names in both cases.

As an example in the “Information Management System of a College” is shown in figure 11.6:

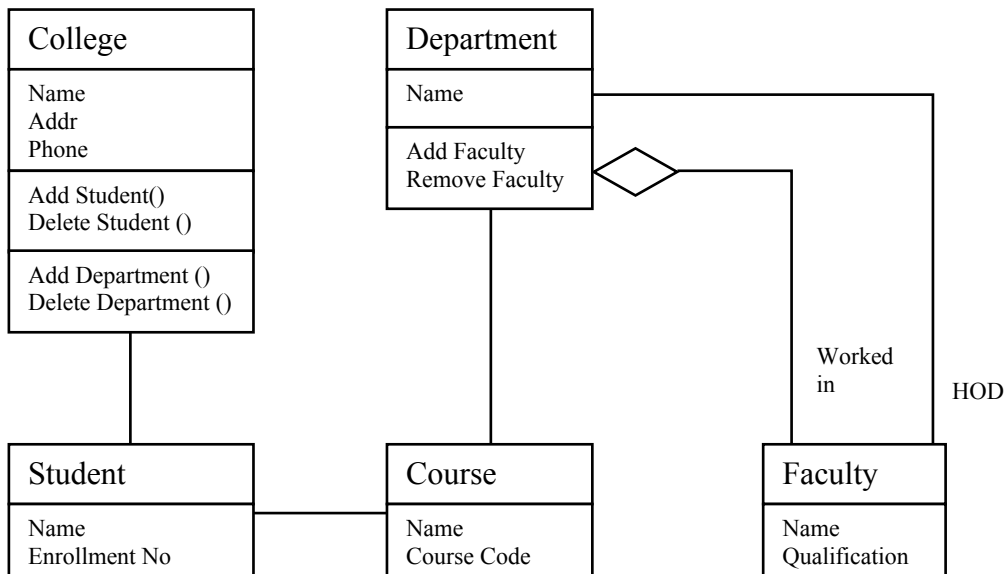


Figure 11.6: Information Management System of a College

- Aggregation represents the special kind of association between the whole and its parts.
- Composite Association: In case there is a strong dependency between class 1 and class 2, as a result, if class 1 is deleted, class 2 is deleted automatically. Use a composite link to show it.

Various constraints of UML Association are:

- Implicit-Relationship between objects is on a concept basis.
- Ordered- Objects can be arranged in some specific way.
- Changeable- Relationship between objects can be modified.
- Add Only- New connection between objects can be added at any time.
- Frozen- Frozen constrained not allowed for modification of already constructed link.

Table 11.1: Difference between Association, Aggregation and Composition

Association	Aggregation	Composition
Denoted by arrow	Denoted by empty Arrowhead.	Denoted by filled Arrowhead.
Can be classified on 1:1, 1:M, M:N	Weak Association.	A strong type of Association.
Association can be in between two or more objects.	Linked Objects can remain in the scope of a system without each other.	Objects cannot remain in the scope without each other.
Objects are Linked.	Objects are not dependent on other objects.	Objects are strongly dependent on other objects.
Example: Relationship between Student and Teacher Object.	Example: Vehicle requires wheel but not every time of the same type.	Example: relationship between file and folder, if folder deleted all files in the folder also deleted automatically.

Analysing Association Traversal

Associations are inherently bidirectional. If the association used in any application is unidirectional, it helps in easy implementation. As the requirement may change with time, there may be a need for a new association that traverses in opposite directions. For the prototype to work, a new behaviour can be added, or bidirectional association may be included. In the case of optimization work, optimize some of the associations.

Associations can be defined as a family of links. It can link multiple classes. It can be classified as *unary*, *binary*, or *ternary relationship*, *aggregation* and *reflexive* (connects the objects of the same classes). Association can be categorized as *weak association* when two classes are linked as if the method of class 1 is using the attribute of class 2 or the return object of class 2. In addition, each association is characterized by three types of cardinality viz. *one-to-one*, *one-to-many*, or *many-to-many*. Association degree tells about the involvement of the number of classes in a link if it is:

- unary-connects same class objects,
- binary-connects two class objects, and
- ternary-connects objects of three or more classes.

One-way Association

- An association which traverses in a single direction will be implemented as a pointer attribute containing the reference to the object.
- In the case of many multiplicities, a set of pointers is required for implementation purposes.
- Use List rather than Set if the “many” end is ordered.
- A dictionary object can be used in the case of multiplicity association, where the dictionary consists of many instances that Map selector values into target values.
- Generally, qualified associations with multiplicity “many” are rare. (It is implemented as a dictionary Set of objects).

Unidirectional Associations

- For implementing unidirectional associations, care should be taken so that uni-directionality is maintained. The implementations for different multiplicity are as follows –
- **Optional Associations:** In this type of association, the class may or may not be linked to each other. It may be possible that customer have a current account or may not be. Consider the case of Customer association with its Current Account.

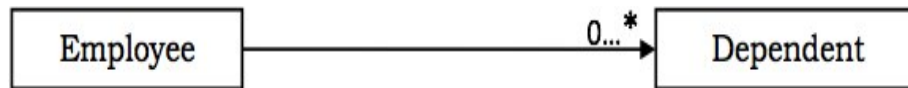


- **One-to-one Associations:** In 1:1 Association, one class instance is connected to just only one instance of the other class. Consider a

situation where Manager is related to its Department in 1:1 relationship.



- **One-to-many Associations:** In this association, single occurrence of a class is linked to many occurrences of other class. In the example below, one department has many employees associated with it.



Bidirectional Association

It is used to relate associations that communicate in both directions regularly on constantly, or it is also defined as an association that relates two objects in both direction. A bidirectional relationship provides navigational access in both directions. There are many implementing ways for these types of associations:

- In the first method, a single side association is implemented, and a searching procedure is required in case opposite side traversal is needed. This is the special case where there is a difference in traversal frequency in both directions. This must be handled by minimizing both types of cost (storing and updating cost); otherwise, the reverse traversal leads to an increase in the cost.
- In the second method, to implement binary association, the pointers may be used for traversal purposes. In this case, if change occurs in any attribute of one class, then it must be updated by another class, so as to maintain consistency but to keep in mind that the number of updates does not cross the number of access.
- In the third method, the relationship between the class is represented by a new association class with the attributes, it is also known as qualified association. Two different objects are used for traversing in both directions.

Note that the association class attribute may be the part of either class in the case of a one-to-one relationship; in the case of one-to-many relationships, attributes are from many objects class, and in many-to-many relationships, it is implemented as a separate class.

In case the association needs some attributes to be shown, then a separate class will be used. The below example shows N:N(many-to-many) association between two classes, and a *new class Work_On* is added containing more details about their association. Figure 11.7 shows the link attribute implemented as a separate association.

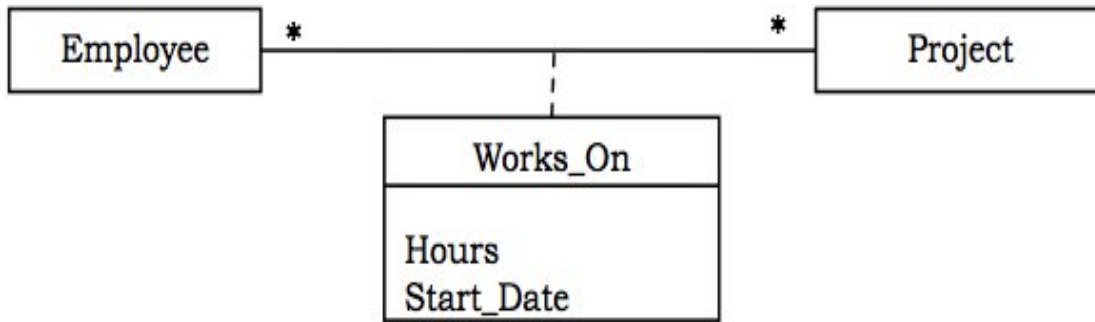


Figure 11.7: Bidirectional Association Between two Classes

Check Your Progress-2

1. Explain the importance of Inheritance. What are various types of Inheritance?

2. Describe association, aggregation, and composition in OO Modeling.

3. Differentiate between specialization and generalization with the help of the example diagram.

11.5 OBJECT REPRESENTATION

Objects represent the static view of object-oriented systems, and their behaviour is defined only in class. Objects should not be defined without the classes. Object representation means representing objects using an object model. Object can be represented diagrammatically. Attributes of the objects can be represented in terms of basic data types viz. integer, float, string etc. in a class. The object designer's task is to decide and identify how to combine and implement objects.

Object Name
+ Object Attribute1
+ Object Attribute2

Object Representation

Rules for representing objects by using the diagram

- Analyze the system and identify all the objects.
- Important instances should be identified.

- It should have some heading.
- Shows some important associations between objects.
- Add comments or notes wherever necessary.

Object diagram consists of objects and link to connect them with other objects.

Object diagram of the Order Management System is shown in figure 11.8 with objects: Customer, Order, Priority order, Normal Order.

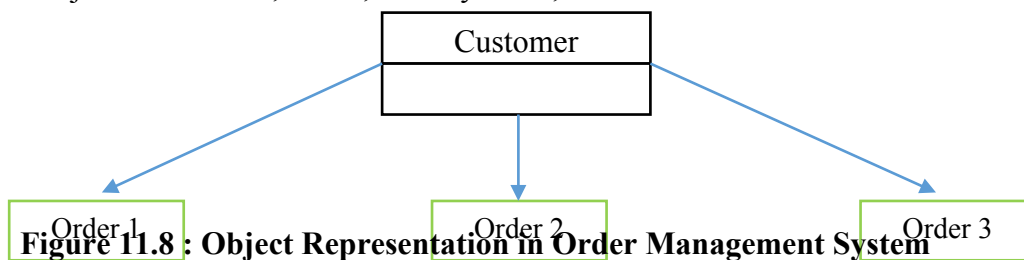


Figure 11.8: Object Representation in Order Management System

Major elements of object model are abstraction, encapsulation, modularity and hierarchy. *Abstraction* is defined as a system specification focuses on some relevant properties for a user or viewer while hiding the details. Class

encapsulate attributes and methods together which can be accessed by the objects. Modularity implies software or program is divided into various modules. Modules should be more cohesive and coupling between them should be less. Hierarchy in oops can be implemented by means of inheritance. There are various benefits of object modelling and representation.

- With the use of objects all oops language features can be accessible.
- Reuse of design and software is possible by using object model.
- Application of object modelling includes problems related with various fields viz. aviation, automation, robotics, banking, business etc. Many complex problems can be easily implemented by using object representation.

There are many ways for implementing or representing objects during designing phase but utmost care should be taken by the designer so as to minimize cost, execution time and memory space required. Consider an example of employee object where the (EIN employee identification number is an attribute which is implemented as an attribute of the employee class or a new class formed for it, as shown in Figure 11.9.

- In First case, object EIN can be represented as an attribute of employee class, it can take any data type either integer or string.
- In second case employee identification number is implemented as an association to employee identification number object which itself can contain either an integer or a string.

There is a trade-off between these two approaches, and it is the choice of the designer to opt for the best one.

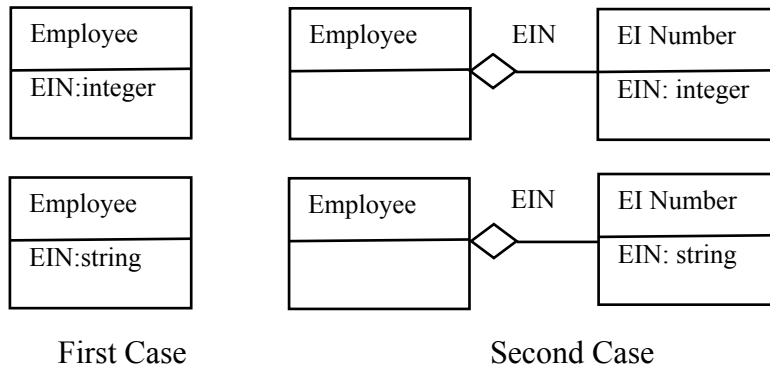


Figure 11.9: Attribute as a part of a class or to form a new class

In the alternate method, the object designer may go for combining classes containing related objects. In the figure 11.10 shown below there are two different implementations of a *line class* one contains all the attributes of a class, and in the other representation, it is embedded as an attribute of the *point class*.

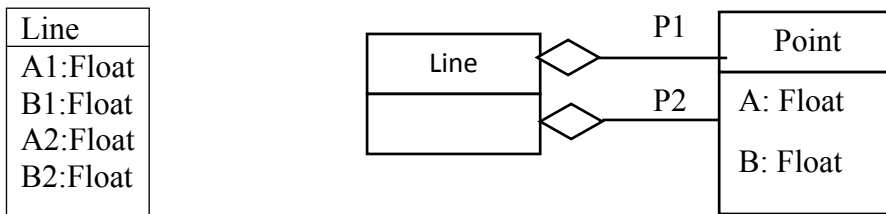


Figure 11.10:Embedded and explicit representation of same Object

Both of these representations will be used by object-oriented designers depending on the choice.

11.6 DESIGN OPTIMIZATION

It considers various design models constructed and trade-offs between time, space, cost and other metrics but does not leave behind other criteria viz. ease of maintaining, implementation, extension, ease of understanding, reusability and many more.

A design model add detailing to support efficient information access. The designer uses the analysis model as a framework for the implementation, as it stores the logical information regarding the system. While designing a system the correctness of the analysis model is required for efficient implementation and then optimize it. Some important points related to design optimization are:

- Often a small part of the code is responsible for most of the time or space costs.
- Optimizing critical areas is the best approach to work on all areas evenly.
- Optimized system is more ambiguous, and there is a little possibility of its reusability.
- It is good to maintain the correct balance between efficiency and transparency.

Tasks that need to be performed for design optimization include:

- Work on providing Efficient access paths.
- Reorder the computations for greater efficacy.
- Intermediate results should be saved to avoid re-computation.

i. Adding Redundant Associations for Efficient Access

As the redundant information is not of any use so rearrangement of associations must be realized to optimize critical aspects of the system. Let's take an example of an employee skills database

Operation organization identify_expertise () returns all those persons in the specified organisation with a given skill set.

Consider an example where the Organization has 1500 employees, each one is expert in 5 tasks then the number of total entries become 7500. To increase the hit ratio for any query certain measures should to be taken. If there is a need to improve the number of hits for any asked query, the classes/objects can be implemented by means of any technique like indexing or hashing which increases the access to frequently retrieved objects. This may increase the cost, and there is a requirement for additional memory to handle indexes or hash table.

While system optimization following points should be considered :

1. Examine each operation and see what associations it must traverse to obtain its information. Identify all unidirectional and multidirectional associations.
2. Next, for each operation, keep the following points into consideration:
3. How Frequently can it be accessed, and what cost to perform it? Compute the *fan-out* along the path through the network. The *fan-out* of a class or method is "the number of other classes used by that class or the number of other methods called by that method". Keep the count of all associations that come across in the track and accordingly calculate the fan-out for the complete path.
4. Nested loops can be avoided, and frequent, costly operations with a low hit ratio can be justified using indexes.

Consider an example of withdrawal operation from the bank ATM machine.

- All the transaction of cash greater than Rs. 50,0000 will take into account and will be informed to the government for further actions.
- The classes involved during this may include
 - ATM to Bank Account,
 - Bank Account to Update Account,
 - All the updates of withdrawal operations are filtered against the checkpoint where cash greater than Rs. 50,000 is withdrawal.
- This can be possible with the derived association from Bank Account to Update Account would speed this operation.

ii. Algorithm Optimization

In object-oriented design, it requires to work on data structure and algorithm both. The step includes to perform are:

- a) The computational order may be changed to increase the efficiency of the algorithm.
- b) Check if the order of executing the loop is correct. If not, then reorder it.
- c) Many paths in the code lead to dead paths, this need to be identified, modified or removed from the algorithm.

iii. Saving Derived Values to Avoid Re-computation

Derived attribute values are based on the values of base class attributes. To avoid inconsistency in the update of derived attributes, three methods are used

- a) Derived attributes are updated back to back as their based attributes are updated.
- b) In another method, as the base class attribute is updated many times and derived attributes are updated once all updation of the base class attribute is complete.
- c) Re-compute derived attribute with the update of each base class attribute. This method is time-consuming and should be avoided.

Reification Behaviour

- Behaviour written in code is rigid; you can execute but cannot manipulate it at run time.
- If you need to store, pass, or modify the behaviour at run time, you should reify it.

11.7 DESIGN DOCUMENTATION

Design documentation is the information that completely describes the design process. This term is derived from an idea regarding documenting the design in formal writing. As OOP's allow the reuse of software content, that is why there is a growing demand to document the existing software. It is good to apply object-oriented technology to documentation also.

Documentation should be precise in explaining the capabilities of already constructed components. It may be divided into various types

- User Documentation
- System Documentation
- Project Documentation

Design documentation is part of the system documentation that addresses all interfaces and implementation aspects of software to facilitate maintenance and reuse. One advantage of reusing is improving the programmer's overall productivity and quality of software.

These design documentation include:

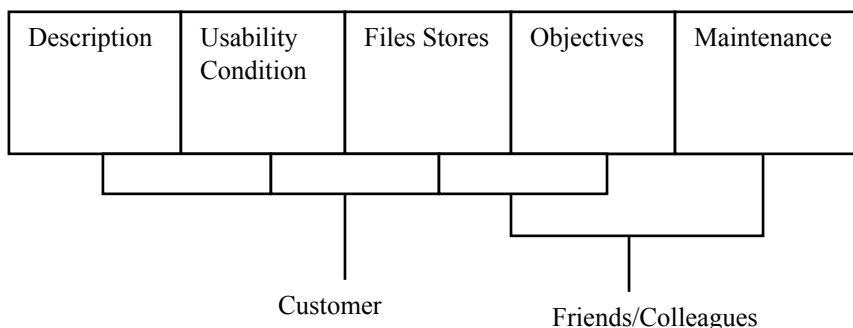
- Information needed to reuse the software design.
- Information required for implementation and maintenance.

The design documents can further be classified as:

- i. Static overview

- ii. Dynamic overview
 - iii. Class interface description
 - iv. Task interface description
 - v. Class implementation
 - vi. Task implementation
- i. Static overview: It contains the details of the required platform, its Software and Hardware requirement, Software Architecture, specify classes, its organization and a description of all classes.
 - ii. Dynamic Overview: It handles all event information, inter-application communication control of flow etc.
 - iii. Class Interface Description: It includes class view, describing its various operations and class capabilities.
 - iv. Task Interface Description: It details the use of classes for new applications or tasks in document. For example, for a particular task which class can be used directly or by applying inheritance property. Which subroutine or behaviour of the class can be overridden etc.
 - v. Class Implementation Description: It may include the internal structure of the class with detail and the purpose of its data and methods. One important point to note here is that the interface description of class is not discussed here.
 - vi. Task Implementation Description: It documents the classes/ methods that are part of a certain task and is usable for maintenance engineers.

Class Documentation



All the design decisions taken by an object designer must be in written form and be documented for future reference. It may be the case that the current designer will shift to another place, and a new person will come to look after the current project. In this case, the new designer will use already documented data of an unaccomplished project to get information on the incomplete project. As in the Analysis phase, the system requirement document is prepared in the same way the design document is shown with a more detailed illustration of the object model containing text and graphical representation, and a description of all the features like association, inheritance, and abstraction etc.

A good design document is significant in many areas like :

- i. There may be a probability that the software may be divided into many modules, and many developers are working on different modules.
- ii. As there is no technique of software development that goes in a sequential manner and it is an iterative process, documentation is necessary.

- iii. Each software is developed in parts or comes in a different version, and it's easy to develop a new version from the previous one if the design is documented.
- iv. Software is tested and evaluated using many performance criteria so it must be documented.
- v. Finally, maintaining software is a very hectic task and requires a document with all the details.

The content of the Design Document may contain

- i. A basic architecture of a complete system showing all the classes, its object and other required details.
- ii. Diagrams for different modules can be included for more details.
- iii. All aspects of the system behaviour in different circumstances need to be address.

Features of a good design document

- i. Highly organized and well structured.
- ii. Summarized, uniform or homogenous, reliable and perfectly framed.
- iii. It should cover all the requirements collected during the analysis phase.
- iv. It should be less Illustrative and more emphasis on images or pictorial representation.

** Check Your Progress-3**

5. Explain the statement: "Abstraction is relative to the perspective of the viewer".

6. Mention five possible attributes and three possible behaviours of the class: Employee, Table, Dishwasher, Aeroplane, Tree.

7. Specify the UML diagram used for Object-Oriented Modelling.

11.8 SUMMARY

Object Oriented Analysis and design is an approach for analysing and designing of a system by applying the concepts of OOP's. The purpose of OOA is to identify all objects of problem domain, their relationship and behaviour and during design their attributes, functions and how they all collaborate or interact. The design phase refines the analysis model and indicates the desirable technology and other implementation constraints. Designer uses processes of function model as an object and events of dynamic model as an operations in object design phase. Design must be extended in iterative manner for best optimization. Classes and methods details need to be adjusted to apply inheritance. Types of association viz. unidirectional or bidirectional association can be identified and implemented. Modularity should be applied so as to give convenience to the programming team. The design model should have all the details required so that programmers can implement the design in to a code. All design decisions related with object design should be documented. Various techniques are used in OOD to make the best model for implementation . This unit explained designing controls, association and design optimization.

11.9 SOLUTION/ANSWER TO CHECK YOUR PROGRESS

Check your progress - 1

1. An object oriented design deals with classes and objects. When some actions are performed on objects they changed their states. Various symbols used in State Transition Diagram are shown in figure 11.12

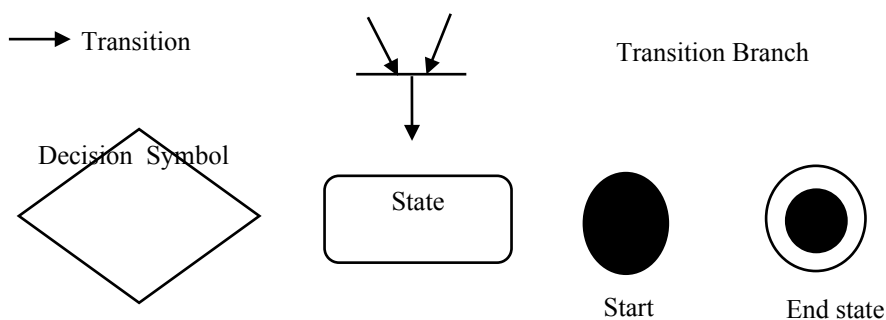


Figure 11.12: Symbols used in State Transition Diagram

State Transition Diagram for searching a friend :Consider a user is in search for another user to make him or her a friend. There may be many events that can occur and result in various states. Step-by-step procedure include for this activity includes:

- Consider an object friend.
- Identify various events or actions.
- Find out the different states.
- Represent all by using STD shown in Figure 11.13

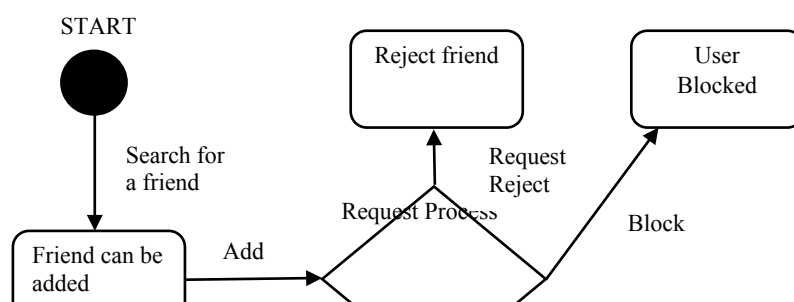


Figure 11.13 : State Transition Diagram for searching a friend

2. Concurrency: A program doing more than one task, that is performing multiple tasks at a time is known as a concurrent program for example, consider a browser viz. get a request for a page; audio playing; showing the bytes for an image; user dialog and so on. Sometimes multiple CPUs are performing these tasks. In some other systems, it can be handled by a time-sharing CPU that is switching from one activity to another. Some examples of concurrent applications include Simulation, Input-Output processing, Mobile code, and Embedded system.

3. State Transition Diagram of ATM Machine shown in Figure 11.14

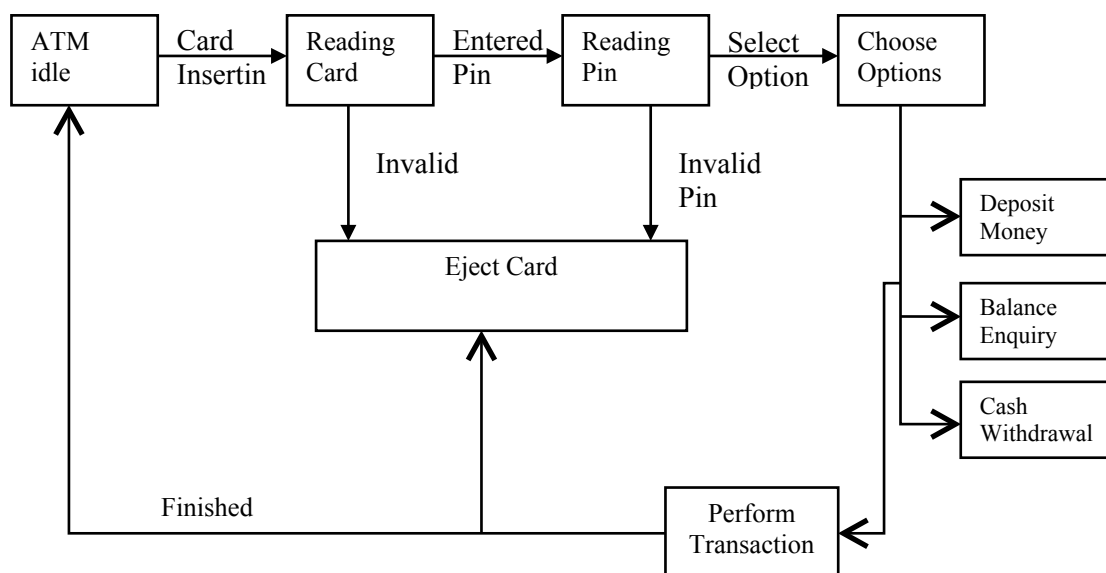


Figure 11.14: ATM Machine State Transition Diagram

☛ Check Your Progress - 2

1. Importance of Inheritance includes

- Reuse: Information can be reused by deriving the new child class from the existing parent class.
- Code Reuse: It helps in saving time and efforts of the programmer.
- Easily Modified: Code can be easily modified.

- **Features Inherited:** All the subclasses of the derived class inherit the features of the base class.

In a nut shell we can say that Inheritance results in expressing the relationship between classes close to real-world model.

Types of Inheritance

- **Single:** In this type of inheritance, one subclass inherits one base class
- **Multiple:** This way of inheriting allows derived class to inherit properties from many base class
- **Hierarchical:** It allows many classes to inherit properties from one base class.
- **Multilevel:** A new derived class inherits an already created derived class.
- **Hybrid:** In a code, if more than one inheritance is used

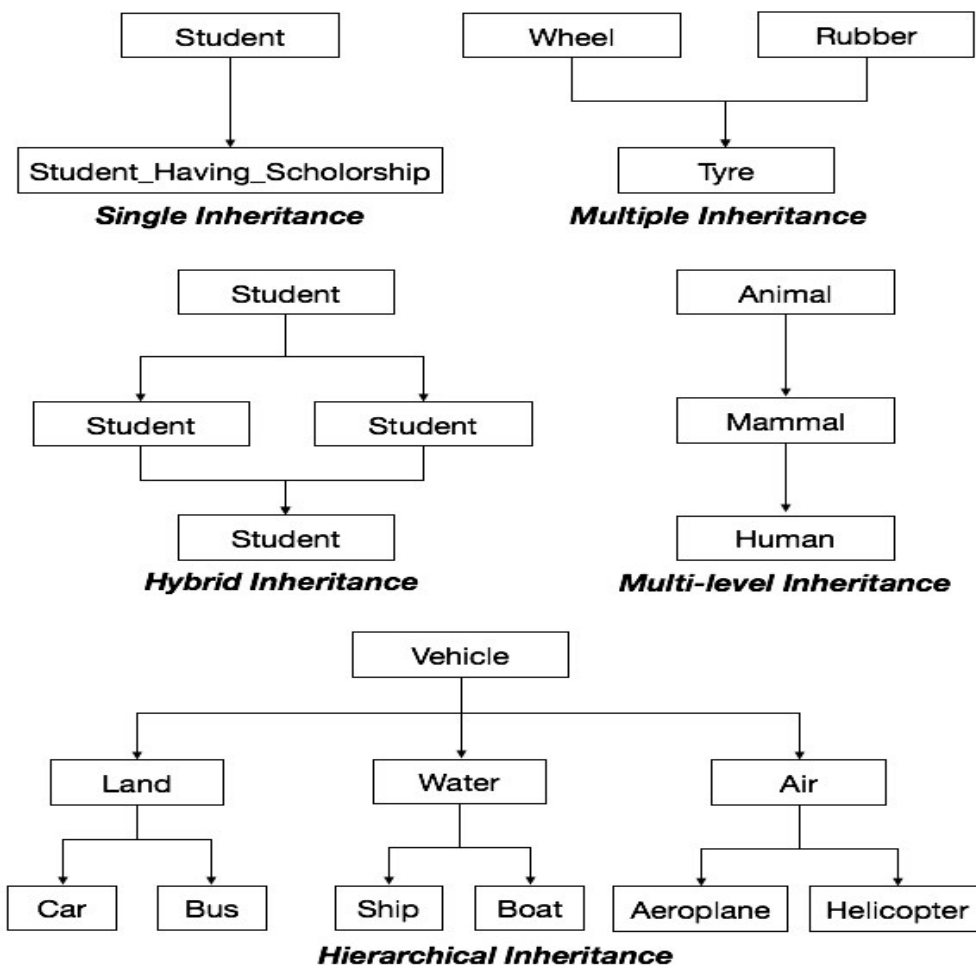


Figure 11.15: Types of Inheritance

2. **Association:** It represents a weak relationship between objects. Consider a relationship between a Doctor object and a Patient object. These two objects are independent and can be created and destroyed independently. A Doctor can be associated with many patients, and one patient can take the advice from many doctors. No one is an owner object. A link or association is

represented by an arrow. Association relationship or cardinality can be one-to-one ,one-to- many and many- to- many.

Aggregation:It is a specialized association among objects. In this case, there exists an ownership. Its characteristics include that the parent or owner can exist alone without a child and vice-versa. For example, an employee may belong to one or many department. If the department is deleted, the object employee still exists. The relationship between an object in an aggregation cannot be reversed.

Department owns Employee, but Employee doesn't own Department.

Composition: In this, the child cannot exist without the parent. It denoted the strong aggregation or death relationship between objects. For example, consider if a house consists of 2 to 3 rooms and if the house gets demolished, then all the rooms are also devastated.

3. Difference between generalization and specialization can be given in tabular form as follows

GENERALIZATION	SPECIALIZATION
Generalization is a Bottom-Up approach.	Specialization is a top-down approach.
In Generalization, the size of the schema gets reduced.	In Specialization, the size of the schema gets increased.
Generalization is usually realised on a group of entities.	Specialization can be applicable to a solo entity.
Generalization can be well-defined as a technique of making groups from several entities.	Specialization is a process of forming subgroups within a single entity.
Generalization takes the union of some lower-level entities to form a higher-level entity.	Specialization is the opposite of Generalization. In Specialization a subset of a higher-level entity is taken to produce lower-level entities.
In Generalization, the variance and resemblances among lower-level entities are overlooked to form a higher entity.	In Specialization, a higher-level entity is fragmented to form subordinate entities.
Inheritance is not used in Generalization.	Inheritance is used in Specialization.

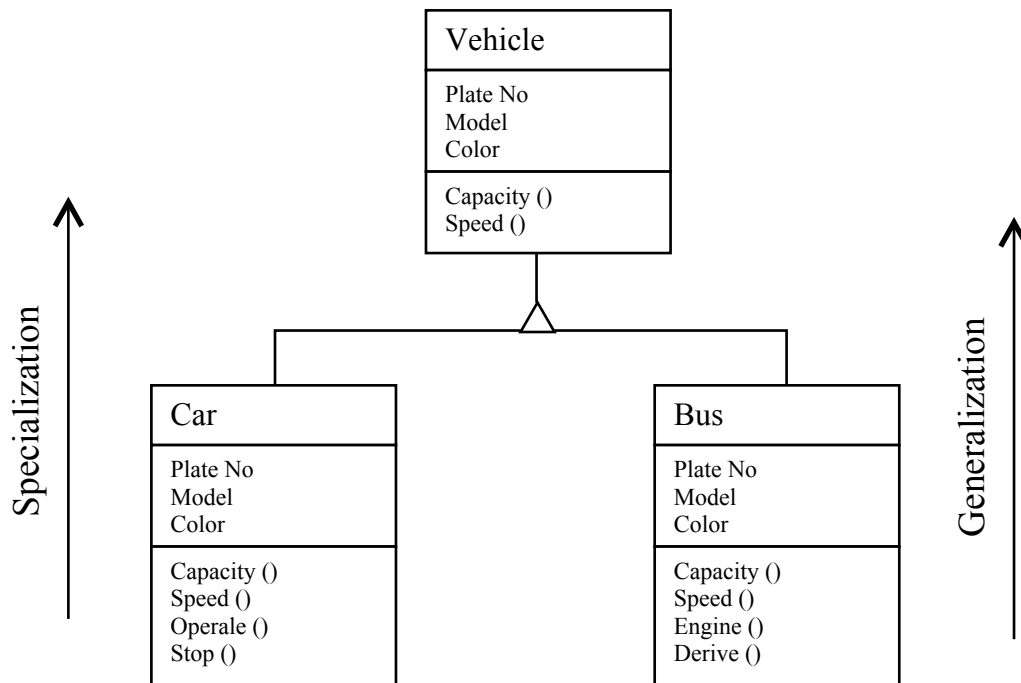


Figure 11.16 : Generalization and Specialization

☛ Check your progress - 3

1. The statement "Abstraction is relative to the perspective of the viewer" can be explained as follows:

It specifies that the abstraction of any entity can be different, there can be different according to the perspective of the user. Consider an example of a Pet Cat. According to the cat's owner, the most important things are milk, bread, cat's house, playing with the cat, bathing the cat, roaming time etc . However, according to veterinary doctor's perspective, the importance should be on cat's health, vaccination, body function, etc. Like this, there can be two abstractions for the cat- one for the cat's owner and the other for the cat's vet.

2. Attributes and behaviours of Some Entities

Employee

Attributes	Behaviours
Employee Name	Leave count
Employee Number	Salary
Employee Pan Number	Evaluate Income Tax
Designation	

Home Address	
--------------	--

Table

Attributes	Behaviours
Material Type	Put belongings on the Table
Texture	Remove belongings from theTable
Weight	Sanitize Table
Height	
Width	

Dish Washer

Attributes	Behaviours
Manufacturer	Start Cleaning Utensil
Model Number	Stop
Time Taken	Load Utensils
Energy Rating	
Warranty	

Aeroplane

Attributes	Behaviours
Airline	Take off
Make	Land

Sitting Capacity	Auto pilot
Engine	
Time Taken	

Tree

Attributes	Behaviours
Scientific Name	Release Oxygen
How much old	Absorb Carbon dioxide
Height	Grow new leaves
Diameter	
Use	

3. The Unified Modeling Language is the most popular and standard modeling language for object-oriented modeling. The most commonly used diagrams are:

- Use case diagram: It demonstrates the interface between a system, and its environment (users or systems) within a certain condition.
- Class diagram: It shows the different objects, their relationship, their functions, and data or attributes.
- Sequence diagram: This perfectly displays the interactions amongst the various objects in the system, and between actors and the system's objects.
- State Machine Diagram: Used to represent the behaviour of a system in response to external and internal events.
- Activity diagram: Represents the data flow among various processes.

11.10 REFERENCES/FURTHER READING

- 1) Grady Booch, James Rumbaugh and Ivar Jacobson, "The Unified Modeling Language User Guide", 2nd Edition, Addison-Wesley Object Technology Series, 2005.
- 2) Grady Booch, Robert A. Maksimchuk, Michael W. Engle, Bobbi J. Young, Jim Conallen, Kelli A. Houston, "Object-Oriented Analysis and Design with Applications," 3rd Edition, Addison-Wesley, 2007.
- 3) Blaha, M. , "Object-Oriented Modeling and Design with UML: For VTU",

Pearson Education India, 2005.

- 4) O'docherty, Mike, "Object-oriented analysis & design", John Wiley & Sons, 2005.